

DOCUMENTO DE ARQUITECTURA
TÉCNICA · V0.2

SECURITY FRAMEWORK

Plano técnico actualizado de un framework de detección de amenazas en tiempo real: sensor ETW en Rust, motor en Go con correlación de kill-chains, riesgo por host, beaconing y umbrales volumétricos, importación Sigma y consola web Next.js. Esta revisión refleja el estado implementado y verificado del repositorio.

Ruby570bocadito · Diseño y arquitectura

Septiembre 2026 · nombre provisional del proyecto

Índice

1. Visión y Filosofía de Diseño	1
1.1 Defensa informada por ofensiva	1
1.2 Qué mejoramos respecto a Fibratus	1
1.3 Del diseño a la implementación	2
2. Arquitectura General del Sistema	3
2.1 Flujo de datos extremo a extremo	3
2.2 Decisión de lenguaje: Go y Rust juntos	4
3. Componentes Principales	5
3.1 Sensor (Rust, Windows ETW) y devsensor	5
3.2 Motor de detección (Go): pipeline y paquetes A1-A4	5
3.3 Consola web y API	6
3.4 Extensiones (filaments, fase futura)	7
4. Modelo de Datos y Esquema de Eventos	7
4.1 Pipeline de enriquecimiento y límites del ingest	9
5. Sistema de Reglas YAML	9
5.1 Operadores soportados	10
5.2 Reglas sembradas desde TTPs ofensivas	11
5.3 Importación de reglas Sigma (A4)	11
5.4 Control de falsos positivos	12
6. Stack Tecnológico y Estructura del Repositorio	12
7. Roadmap de Desarrollo	15
8. Plan de Acción y Verificación	16
8.1 Ejecutar el tracer bullet	16
8.2 Verificación continua	17

1. Visión y Filosofía de Diseño

Este documento define la arquitectura técnica de un framework open source de detección de amenazas en tiempo real, concebido como alternativa moderna a Fibratus. El proyecto combina un sensor de eventos del sistema anclado en ETW con un motor de detección de comportamiento, correlación de secuencias y una consola web interactiva de triaje. El nombre del proyecto es provisional (security-framework) y se mantendrá hasta que la comunidad elija la marca definitiva. A diferencia de la v0.1, que describía una promesa de diseño, esta revisión v0.2 describe el sistema tal como está implementado y verificado hoy: cada afirmación de este texto corresponde a código en el repositorio, con pruebas de unidad, E2E sobre binarios reales y mediciones de rendimiento publicadas.

1.1 Defensa informada por ofensiva

La ventaja competitiva central del proyecto no es tecnológica sino cognitiva: quien diseña las detecciones ha operado del otro lado del tablero. Un red teamer sabe qué ruido produce Cobalt Strike al inyectar, cómo se comporta Mimikatz al abrir LSASS, y qué cadena de procesos deja tras de sí un phishing con PowerShell ofuscado. Ese conocimiento convierte las reglas de detección en algo cercano al comportamiento real de un adversario, no a patrones teóricos de laboratorio. Cada módulo del framework se plantea desde la pregunta: ¿cómo intentaré evadir esto en mi próxima operación, y cómo lo cierro?

De esta filosofía se derivan tres principios de diseño que gobiernan todas las decisiones de esta arquitectura. Primero, el comportamiento importa más que las firmas: las Indicators of Compromise caducan, las TTPs persisten. Segundo, la telemetría debe ser resistente a la evasión: el sensor se ancla en ETW user-mode y no en hooks de user-mode que un malware pueda desinstalar. Tercero, la usabilidad es una feature de seguridad: una herramienta que nadie mira por complicada no detecta nada.

1.2 Qué mejoramos respecto a Fibratus

Fibratus es un excelente punto de referencia: su motor de reglas, su soporte ETW y su integración YARA demuestran madurez técnica. El análisis previo del proyecto identificó seis áreas donde este framework puede diferenciarse de forma clara. La tabla siguiente resume el posicionamiento objetivo, que el roadmap del capítulo 7 convierte en entregables concretos y que la columna de estado de ese mismo capítulo contrasta con la realidad.

Área	Fibratus (estado actual)	Enfoque de este framework
Rendimiento	Bueno en cargas normales, con degradación bajo presión extrema de eventos.	Pipeline con buffers acotados, capas de estado con límites duros y evaluación O(1) por regla indexada; latencia p99 medida en CI con bench nocturno.
Integración SIEM/SOAR	Exportación por formatos, sin conectores plug-and-play.	Webhook de alertas con cola acotada y token Bearer, export JSONL/CSV con neutralización de inyección de fórmulas, API REST documentada con OpenAPI.
Experiencia de usuario	Interfaz principalmente CLI.	Consola web Next.js con triaje en vivo, gestión visual de supresiones y dashboard de operaciones con telemetría real.
Threat hunting	Detección reactiva basada en reglas conocidas.	Reglas sembradas desde TTPs ofensivas validadas en laboratorio, correlación multi-etapa y detección volumétrica y de beaconing.
Contenedores y K8s	Soporte limitado en el núcleo.	Collector eBPF consciente de namespaces y cgroups planificado en fase 2 (aún no implementado).
Interoperabilidad de reglas	Formato propio cerrado.	Importador Sigma determinista con procedencia preservada y validación fail-loud por regla.

Tabla 1. Comparativa de posicionamiento frente a Fibratus por área funcional.

0,4 ms p99 medido de ingest→alerta en loopback (bench, 2 000 eventos)	17 + 4 operadores de reglas (11 + familia i*) + detectores conductuales y volumétricos	0 dependencias de driver de kernel (ETW user-mode, sin Sysmon obligatorio)
---	---	--

1.3 Del diseño a la implementación

La v0.1 de este documento se escribió antes del primer commit y definía el norte. Desde entonces, el desarrollo por rondas ha aterrizado y verificado el núcleo completo: el pipeline de ingesta con autenticación por token compartido y rotación, el motor de reglas con recarga en caliente, el correlador de kill-chains, el detector de beaconing, el detector de umbrales volumétricos, el scoring de riesgo por host, el importador Sigma, la persistencia SQLite opcional, el webhook de alertas, la exportación JSONL/CSV y la consola web de operaciones. El roadmap interno registra 7 de 17 líneas cerradas, y los cuatro paquetes de detección (A1 a A4) tienen el ciclo de vida completo: aterrizaje, verificación en caliente por el Director y auditoría cruzada del rol de seguridad.

Esta revisión mantiene la estructura de capítulos de la v0.1 para que la lectura comparativa sea directa, pero corrige tres tipos de desviación que docs/README.md documentaba con honestidad: el stack de la consola (Next.js 16 en lugar de React + Vite), la API del motor (net/http de stdlib en lugar de Gin, con OpenAPI mantenido a mano y validado contra el motor vivo) y el conjunto real de operadores y tipos de evento. Todo lo que la v0.1 anunciaba como fase futura y sigue sin implementar (YARA, gRPC, filaments Python, eBPF) se

declara explícitamente como pendiente en el capítulo 7, sin presentarlo como capacidad existente.

2. Arquitectura General del Sistema

La arquitectura sigue un modelo de tres planos desacoplados: recolección (sensor), análisis (motor) y consumo (interfaces). Cada plano escala y falla de forma independiente: si la consola web se cae, el motor sigue detectando; si el motor se reinicia, el sensor almacena en buffer y reintenta. Esta separación permite distribuir los componentes: el sensor vive en cada endpoint, mientras el motor y las interfaces pueden centralizarse en un servidor de análisis. La Figura 1 muestra los estratos funcionales y el flujo de datos de arriba abajo.

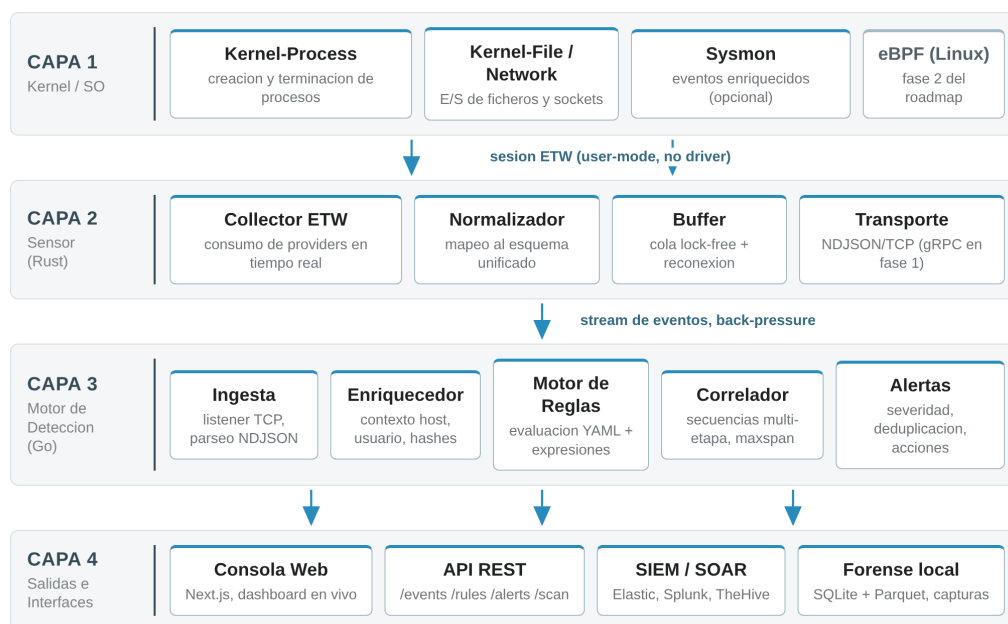


Figura 1. Arquitectura por capas del framework: sensor, transporte, motor de detección y consumo.

2.1 Flujo de datos extremo a extremo

El recorrido de un evento es siempre el mismo. El kernel emite notificaciones a través de los proveedores ETW de Windows (Kernel-Process y opcionalmente Sysmon como ruta de ingesta alternativa). El sensor Rust las consume en tiempo real mediante una sesión de trazado en user-mode, sin driver propietario, y las traduce al esquema de eventos unificado del capítulo 4. Un buffer acotado absorbe los picos de telemetría y el transporte las envía por TCP en formato NDJSON al puerto de ingesta (127.0.0.1:7777 por defecto). Desde la primera conexión, el sensor debe autenticarse con el token compartido del despliegue (comando AUTH sobre el canal, comparación en tiempo constante); la rotación del token está soportada nativamente aceptando el token previo durante una ventana de transición, de modo que una rotación de credenciales no requiere reinicio coordinado de la flota.

En el motor, cada evento pasa por la ingesta (parseo, validación y truncado de campos en el punto único de normalización), el enriquecedor (contexto de host y usuario, añadido en un mapa aparte que nunca muta la evidencia cruda) y el motor de reglas, que evalúa las condiciones YAML indexadas por tipo de evento y recargadas desde disco cada quince segundos. Sobre esa columna vertebral operan tres detectores adicionales: el correlador de secuencias multi-etapa por host, el detector de beaconing sobre la regularidad de las conexiones salientes y el detector de umbrales volumétricos por ventana fija. El scoring de riesgo agrega la severidad de lo detectado por host con decaimiento temporal. Las alertas resultantes se publican simultáneamente en la API HTTP local (puerto 7778, con stream SSE para la consola), el webhook configurable hacia SIEM/SOAR, el almacén SQLite opcional y los comandos de exportación JSONL/CSV. El canal de salida aplica además deduplicación, supresiones operador-local con expiración y estados de triaje persistentes.

2.2 Decisión de lenguaje: Go y Rust juntos

La combinación Go más Rust asigna a cada lenguaje el trabajo para el que es mejor. Rust protege el plano caliente del sensor: consume ETW a decenas de miles de eventos por segundo sin pausas de garbage collector y con garantías de seguridad de memoria en un componente que corre con privilegios altos; el binario se compila con LTO y strip, y se niega a ejecutarse fuera de Windows en lugar de inventar telemetría. Go acelera el desarrollo del motor y sus interfaces: goroutines y channels modelan naturalmente el pipeline de eventos, y la elección deliberada de la stdlib (net/http para la API, cobra para la CLI, bubbletea para el panel interactivo) mantiene el binario único, sin CGO y trivial de desplegar. El contrato entre ambos mundos es el esquema de eventos y el protocolo de transporte, no el código compartido, lo que mantiene los binarios independientes.

Componente	Lenguaje	Justificación técnica
Sensor / collector	Rust	Sin pausas de GC en el camino crítico de telemetría; ferrisetw para las sesiones ETW (dependencia solo-Windows); binario estático con LTO y strip para despliegue masivo; honestidad de plataforma: rechaza ejecutarse donde no hay telemetría real.
Motor de detección	Go	Concurrencia nativa (channels/select) para el pipeline; binario único CGO-free; stdlib net/http para la API; cobra para la CLI; modernc.org/sqlite (driver puro) para el store opcional; bubbletea/lipgloss para el panel interactivo.
Consola web	TypeScript	Next.js 16 + React 19 + Tailwind 4 para la cabina de operaciones; el hub console-service (Bun + socket.io) hace de puente con estado de sesión hacia la API del motor.
Extensiones (filaments)	Python	Runtime embebido con sandbox para scripts de análisis en fases futuras; la comunidad de analistas vive en Python. No implementado aún (fase 4).

Tabla 2. Asignación de lenguaje por componente y su justificación.

3. Componentes Principales

Este capítulo describe las responsabilidades internas de cada bloque de la Figura 1. El criterio de reparto es simple: un componente, una razón para cambiar. El sensor solo habla con el kernel y el transporte; el motor solo habla con eventos, reglas y detectores; las interfaces solo hablan con alertas y datos. Los quince paquetes internos de Go delimitan esas fronteras, y cada paquete nuevo que introduce estado de larga vida hereda las mismas obligaciones: límites duros de memoria, telemetría de sus contadores y pruebas de contención con -race.

3.1 Sensor (Rust, Windows ETW) y devsensor

El sensor abre una sesión ETW por proveedor y consume los callbacks en hilos dedicados. Cada registro se mapea al esquema unificado sin enriquecimiento pesado: el sensor es rápido y tonto a propósito, porque cada microsegundo que gasta aquí es un microsegundo robado al endpoint monitorizado. Un buffer de capacidad fija absorbe picos; si el consumidor remoto se cae, el sensor aplica back-pressure, marca la pérdida y se reconecta con reintento exponencial. La misma honestidad de plataforma se aplica al empaquetado: el crate compila en cualquier sistema operativo (normalización y transporte son portables), pero el binario real se niega a arrancar fuera de Windows en lugar de fabricar datos. Junto al sensor real, el repositorio mantiene devsensor, un generador de eventos para desarrollo y verificación que reproduce escenarios ofensivos canónicos y admite un modo de ráfaga (-burst) para ejercitar los umbrales volumétricos; todo fixture sintético está etiquetado como demo en la propia consola, de modo que nunca se confunde telemetría de laboratorio con evidencia de producción.

3.2 Motor de detección (Go): pipeline y paquetes A1-A4

El motor se organiza en cinco etapas encadenadas por canales tipados. La ingesta acepta conexiones de múltiples sensores, valida cada evento contra el esquema y aplica los techos de truncado en el punto único de normalización de identidad. El enriquecedor añade contexto sin mutar la evidencia. El motor de reglas mantiene un índice por tipo de evento y evalúa solo las reglas candidatas. El correlador construye secuencias con ventana temporal (maxspan) para ataques multi-etapa. El gestor de alertas deduplica, asigna severidad y dispara las acciones configuradas: log estructurado, notificación a la consola vía SSE, webhook o persistencia en el store. Sobre ese esqueleto, el roadmap del propietario organizó cuatro paquetes de detección con ciclo de vida completo, resumidos en la tabla 3.

Paquete	Nombre	Qué añade	Verificación
A1	Riesgo por host	Score de riesgo decaído por host, ponderado por severidad con vida media de 30 minutos; alimenta el KPI de hosts calientes en stats, métricas Prometheus y consola.	E2E propia + auditoría cruzada de 04; -race limpia.
A2	Umbrales volumétricos	Detección de ventana fija por clave (regla, host, valor de agrupación) con cuota híbrida de estado (8 192 claves globales, 2 048 por regla), expulsión weakest-first, cooldown por clave y recarga en caliente; nunca alimenta el correlador.	E2E 12/12 sobre binarios reales; dictamen vinculante de 04.
A3	Beaconing (C2)	Detector de regularidad temporal (coeficiente de variación) sobre network.connect con perfiles conservadores, anillo acotado y cooldown; chokepoint del ingest con techo de destino (253 runes).	E2E 14/14; cross-review con 3 defectos corregidos.
A4	Importación Sigma	Convertidor determinista Sigma → formato nativo (mapeo de 19 campos, 7 categorías de logsource, gramática de condiciones and/or/1-of/all-of), fail-loud por regla y procedencia preservada.	Round-trip con el cargador de reglas; E2E 10/10.

Tabla 3. Paquetes de detección del roadmap con su verificación (todos aterrizados).

3.3 Consola web y API

La consola es la gran diferencia de experiencia frente a la CLI y está construida como una cabina de operaciones sobre Next.js 16 + React 19 + Tailwind 4. Sus vistas cubren el ciclo completo del analista: dashboard en vivo con siete KPIs y línea de actividad del sensor, cola de alertas con triaje (reconocida, cerrada y notas que se persisten en el fichero de lifecycle del motor), gestor de reglas cargadas con sus tags ATT&CK;, vista de cadenas detectadas por el correlador y vista de supresiones con cuenta atrás de expiración. El menú de exportación descarga la evidencia en JSONL o CSV con neutralización de inyección de fórmulas. La densidad visual es deliberada: un SOC que mira la pantalla ocho horas al día necesita información, no decoración, y por eso los efectos ambientales (rejilla de puntos, texto que se descifra, tarjetas con halo bajo el puntero) son sutiles, respetan prefers-reduced-motion y nunca compiten con los datos.

Arquitectónicamente, la consola no habla con el motor directamente: el hub console-service (Bun + socket.io) media la sesión, agrega el stream de alertas y expone una única superficie de tiempo real al navegador, lo que permite reconexiones limpias y desplegar la consola sin exponer el motor más allá de su API local. La API REST del motor (net/http, puerto 7778) expone los mismos datos con un spec OpenAPI (docs/api/openapi.yaml) que un guard de CI valida contra el motor vivo en cada ronda: hoy son 12 rutas y 29 campos de estadísticas con paridad exacta entre /api/stats y /metrics. La Figura 2 muestra el dashboard con telemetría real del sensor de desarrollo.

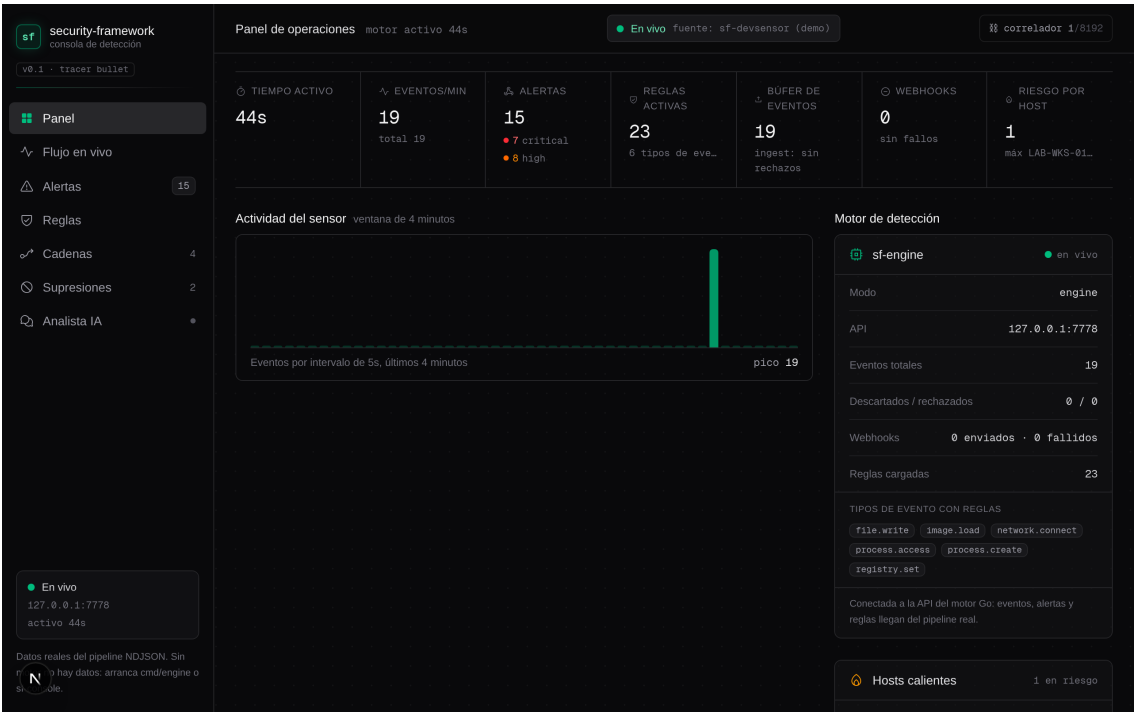


Figura 2. Dashboard de operaciones de la consola: KPIs, actividad del sensor y cola de triaje en vivo.

3.4 Extensiones (filaments, fase futura)

Los filaments heredan la idea de Fibratus de extensiones en Python, con dos mejoras de diseño: ejecución en un entorno aislado con recursos limitados y una API de host que exponga el estado del motor (consultar eventos recientes, pedir enriquecimiento) en lugar de solo callbacks. Cada filament declarará sus triggers por tipo de evento y devolverá hallazgos con el mismo formato que las reglas nativas, de forma que la consola y los conectores no distingan su origen. Este bloque se documenta aquí por honestidad de diseño, no por capacidad: forma parte de la fase 4 del roadmap y no existe código suyo en el repositorio, porque el proyecto no entrega funciones simuladas.

4. Modelo de Datos y Esquema de Eventos

El esquema de eventos es el contrato maestro del sistema: lo emiten los sensores, lo validan las ingestas, lo indexan las reglas y lo consumen las interfaces. Se diseña inspirado en los estándares OCSF y ECS para que las integraciones SIEM sean casi gratuitas, pero reducido a los campos que la detección realmente usa. Cada evento es una línea JSON independiente, lo que hace el transporte trivial (NDJSON), el archivo forense lineal y el parseo paralelo sencillo. La tabla 4 resume los campos vigentes; respecto a la v0.1, el esquema incorpora el par proceso-objetivo y el mapa de enriquecimiento separado de la evidencia.

Campo	Tipo	Descripción
id	uuid	Identificador único del evento generado por el sensor.

Campo	Tipo	Descripción
timestamp	RFC3339	Momento de alta resolución del evento en el kernel.
type	enum	process.create, process.terminate, process.access, file.write, network.connect, image.load, registry.set (7 tipos).
source	enum	etw, sysmon, devsensor, api: origen de la telemetría.
host	string	Hostname del endpoint monitorizado (normalizado y truncado en el ingest).
user	string	Usuario en cuyo contexto se originó la actividad.
process.*	objeto	pid, ppid, name, command_line, image y hashes del proceso; base de los árboles de proceso.
target.* / access.*	objeto	Proceso objetivo y detalles de acceso en eventos process.access (p. ej. apertura de LSASS).
network.*	objeto	L4 del evento: protocolo, IP y puerto origen/destino, dominio resuelto (techo de 253 caracteres).
file.*	objeto	Ruta, extensión y hashes en eventos de fichero.
registry.*	objeto	Clave y valor en eventos de registro de Windows.
tags	lista	Etiquetas libres: attack.t1059.001, hunting, demo, filament:nombre.
enrichment	mapa	Contexto añadido por el motor (nunca por sensores); vive aparte y nunca muta los campos de evidencia cruda.

Tabla 4. Campos del esquema de eventos unificado (v0.2).

```

{
  "id": "3f2a1c9e-8b4d-4f7a-9c21-7d0e5b6a1f88",
  "timestamp": "2026-09-30T14:22:31.482119Z",
  "type": "process.create",
  "source": "devsensor",
  "host": "LAB-WKS-01",
  "user": "CORP\\jdoe",
  "process": {
    "pid": 6612, "ppid": 4820,
    "name": "powershell.exe",
    "command_line": "powershell.exe -nop -w hidden -enc SQBFAFgA...",
    "image": "C:\\Windows\\System32\\WindowsPowerShell\\v1.0\\powershell.exe"
  },
  "tags": ["demo", "parent:winword.exe"]
}

```

Ejemplo 1. Evento process.create en formato NDJSON (transporte y archivo forense).

4.1 Pipeline de enriquecimiento y límites del ingest

El enriquecimiento ocurre después de la ingesta y antes de la evaluación de reglas, de modo que las reglas pueden condicionar sobre campos enriquecidos igual que sobre campos crudos. Los pasos son idempotentes y configurables: resolución de usuario y host y anotaciones de contexto que aterrizan en el mapa enrichment. Cada paso añade información sin mutar los campos originales del sensor, preservando la evidencia cruda para el forense. El ingest es además el chokepoint de normalización de identidad: allí se truncan los campos que los detectores fijan como claves de estado (host, usuario y destino de red con techo de 253 caracteres, suficiente para cualquier dominio RFC 1123 o IP textual), de modo que un feed hostil no puede anclar memoria arbitraria en el estado de larga vida de los detectores. Los pasos costosos trabajan con caché y TTL, porque el enriquecimiento nunca puede ser el cuello de botella del pipeline de detección.

5. Sistema de Reglas YAML

El formato de reglas es la interfaz de usuario más importante del framework: es lo que un analista blue team escribe, lee y mantiene todos los días. Se elige YAML por legibilidad y compatibilidad con control de versiones, y se reserva la expresividad para un conjunto pequeño y predecible de operadores. Una regla declara a qué tipo de evento aplica, qué condiciones debe cumplir el evento, qué acciones disparar al detectar y qué etiquetas MITRE ATT&CK; la caracterizan. Las reglas se cargan de un directorio, se validan al arrancar y se recargan en caliente al detectar cambios en disco (ciclo de quince segundos, configurable), sin reiniciar el motor ni perder el estado de los detectores.

```
# rules/windows/powershell_encoded.yaml
# T1059.001 - PowerShell with encoded command
# Validated in lab: fires on powershell.exe -nop -w hidden -enc .
- name: "PowerShell con comando codificado"
  id: "9f31c2a4-5d7b-4e18-8a02-3b9c6d1e7f40"
  description: >
    Detecta PowerShell con comandos Base64 (-enc / -EncodedCommand) o
    ventana oculta (-w hidden), patron habitual en phishing.
  severity: high
  event_type: process.create
  conditions:
    - field: process.name
      operator: eq
      value: "powershell.exe"
    - field: process.command_line
      operator: contains_any
      value: ["-enc", "-EncodedCommand", "-w hidden"]
  actions:
    - type: alert
      config:
        message: "PowerShell con comando codificado en {host} por {user}"
  tags: ["attack.t1059.001", "attack.execution"]
  enabled: true
```

Ejemplo 2. Regla real del pack sembrado (T1059.001, PowerShell codificado).

5.1 Operadores soportados

El conjunto de operadores es deliberadamente minimalista: cada operador nuevo añade superficie de testeo y de evasión semántica. Con la familia siguiente se cubre la práctica totalidad de las detecciones orientadas a comportamiento, desde cadenas de procesos hasta acceso a rutas sensibles. Respecto a la v0.1 de este documento, la lista refleja los once operadores realmente implementados (el comparador de rango entre acotado se resolvió con las combinaciones de mayor/menor); los operadores de secuencia pertenecen al correlador y no al evaluador puntual, y se describen en la misma tabla por referencia única. La semántica de estos operadores es fuente única: la exportada por `rules.NewMatcher` (que consumen el importador Sigma y los umbrales) comparte el mismo código que evalúa el motor, con pruebas de paridad que impiden divergencias.

Operador	Aplica a	Semántica y ejemplo de uso
eq / neq	cualquier campo	Igualdad o desigualdad exacta. <code>process.name eq "mimikatz.exe"</code> ; <code>process.parent.name neq "explorer.exe"</code> .
contains / contains_any	strings	Subcadena o lista de subcadenas en OR. <code>command_line contains_any ["-enc", "-w hidden"]</code> .
startswith / endswith	strings	Prefijo o sufijo. <code>image endswith "\\rundll32.exe"</code> .

Operador	Aplica a	Semántica y ejemplo de uso
regex	strings	Expresión regular RE2 validada al cargar la regla; longitud limitada para evitar ReDoS.
in / not_in	listas	Pertenencia a lista estática o dinámica (feed TI). user in service_accounts.
gt / lt	numéricos	Comparación para puertos, tamaños o conteos. network.destination.port gt 4444.
i* (ieq, icontains, icontains_any, istartswith, iendswith, iin)	strings / listas	Variantes insensibles a mayúsculas de igualdad, subcadena, prefijo, sufijo y pertenencia; llegan con la importación Sigma para firmas de caso arbitrario.
sequence + maxspan	correlación	Orden de eventos dentro de una ventana temporal por host: process.create de winword.exe seguida de network.connect externa en maxspan 10s.

Tabla 5. Operadores del motor de reglas y del correlador (v0.2, 17 operadores implementados).

5.2 Reglas sembradas desde TTPs ofensivas

El pack inicial no nace de catálogos genéricos: cada una de las 23 reglas (en 6 ficheros bajo rules/windows/) corresponde a una técnica que el autor ha ejecutado o defendido en operaciones reales. Este origen ofensivo define el detalle de las condiciones, que apuntan a combinaciones de campos y no a nombres de binarios aislados, y por tanto resisten cambios triviales de nombre o firma. El pack cubre ejecución ofuscada de PowerShell, descarga con binarios LOLBAS (certutil, bitsadmin), volcado de LSASS vía comsvcs.dll, movimiento lateral con credenciales, manipulación de persistencia y una regla de host en tiempo real para patrones de laboratorio.

Cada regla documenta en su descripción cómo fue validada: qué TTP se ejecutó en laboratorio, qué eventos generó y qué ruido produce en un host limpio. Esta disciplina de evidencia, heredada de la práctica ofensiva, es lo que permite que la comunidad confíe y contribuya packs completos por técnica MITRE. El comando engine validate reporta el estado del pack (OK, avisos y errores) antes de desplegar, y engine rules presenta la tabla de reglas cargadas con severidad, tipo de evento y tags, de modo que el operador siempre puede auditar qué está activo y por qué.

5.3 Importación de reglas Sigma (A4)

El ecosistema Sigma es el estándar de facto de las reglas de detección comunitarias, y reescribirlas a mano es el mayor coste de adopción de cualquier motor nuevo. El subcomando engine sigma convierte directivas Sigma al formato nativo de forma determinista: mismo input produce byte a byte el mismo output, lo que hace las conversiones revisables en diff. El traductor mapea los 19 campos del esquema Sigma, resuelve 7 categorías de logsource hacia los tipos de evento del motor, traduce la tabla de wildcards a los operadores nativos con anclaje de regex cuando procede, y soporta la gramática de condiciones and/or/1-of/all-of, fusionando los OR de un mismo campo y dividiendo los OR entre campos distintos en reglas separadas.

La política de errores es fail-loud y por regla: una directiva con una selección no soportada se omite con un motivo accionable en el informe, y un fichero roto o desmesurado es un error duro, nunca una conversión silenciosamente incompleta. Los techos de superficie (512 reglas de salida, 64 selecciones, 32 campos y 4 MiB por directiva) acotan el recurso consumido por un lote hostil. La procedencia se preserva: el identificador UUID original y la etiqueta sigma viajan en la regla convertida, y el proveedor queda anotado en la descripción. La verificación incluye una prueba de ida y vuelta: cada regla emitida se recarga con el cargador nativo del motor y dispara contra el evento de laboratorio de su TTP; el E2E completo pasa 10/10 sobre binarios reales.

5.4 Control de falsos positivos

Un detector que despierta al SOC cada diez minutos termina silenciado, y un silenciador sin control termina ocultando ataques reales. El canal de salida del motor aplica, en orden: deduplicación por identidad de alerta, supresiones operador-local (pares regla-host con motivo y expiración opcional, cargadas de un YAML gitignoreado y armadas también por API con escritura auditada), límites conservadores por defecto en los detectores volumétricos (severidad medium en el pack de umbrales) y estados de triaje persistidos para que el trabajo del analista sobreviva a los reinicios. La guía docs/false-positive-control.md documenta la receta completa, incluida la opción final de filtrar en el receptor del webhook. La escritura de supresiones por API exige token y se niega en arranques sin él, porque un endpoint de escritura sin credenciales sería un atajo para un atacante que ya esté dentro de la máquina del motor.

6. Stack Tecnológico y Estructura del Repositorio

El stack se elige para minimizar fricción en las primeras semanas y para escalar después sin reescrituras. Se priorizan librerías mantenidas, con licencia permisiva y trazabilidad de rendimiento publicada. La tabla siguiente fija la decisión técnica por capa del sistema tal como está hoy; las desviaciones respecto a la v0.1 (stdlib en lugar de Gin y expr, driver SQLite puro en lugar de go-sqlite3, Next.js en lugar de Vite) se resolvieron a favor de menos dependencias y un binario único sin CGO.

Capa	Tecnología	Motivo de la elección
Sensor (Rust)	ferrisetw, serde, anyhow (LTO + strip)	Sesiones ETW simplificadas; binario estático y pequeño; dependencias ETW confinadas a Windows con compilación portable del resto.
Motor (Go 1.22)	stdlib net/http, cobra, bubbletea/lipgloss, yaml.v3, modernc.org/sqlite	Binario único CGO-free; CLI con subcomandos y ayuda generada; TUI para el panel interactivo; SQLite puro para el forense opcional sin cgo en CI.
Transporte	NDJSON sobre TCP + AUTH de token compartido	Depurable con netcat desde el día uno; rotación de token sin reinicio coordinado; gRPC queda como fase futura si la flota lo exige.
Consola web	Next.js 16, React 19, Tailwind 4, socket.io (hub Bun)	Cabina de operaciones con tipado estricto; el hub media la sesión en tiempo real y evita exponer el motor más allá de su API local.
Almacenamiento	SQLite (opt-in, WAL) + export JSONL/CSV	Evidencia en endpoint sin instalación; retención con poda automática (72 h por defecto); export con neutralización de inyección de fórmulas.
Build y CI	Make, GitHub Actions (SHA-pinned), Docker	Un comando para compilar todo; guard del OpenAPI contra el motor vivo; bench nocturno con presupuesto de latencia.

Tabla 6. Stack tecnológico por capa con su justificación (v0.2).

```

security-framework/
├─ cmd/
│   ├─ engine/           # motor: run, rules, validate, sigma, version, TUI
│   ├─ devsensor/        # sensor de desarrollo (escenarios + -burst)
│   └─ bench/            # arnés de carga y latencia (p50/p90/p99)
├─ internal/             # 15 paquetes delimitados por frontera
│   ├─ ingest/           # TCP NDJSON, AUTH, chokepoint de normalización
│   ├─ enrich/           # enriquecimiento sin mutar evidencia
│   ├─ rules/            # parser YAML, índice, evaluador, NewMatcher
│   ├─ correlate/        # secuencias multi-etapa (maxspan, cap de estado)
│   ├─ beacon/           # A3: regularidad C2 sobre network.connect
│   ├─ threshold/        # A2: umbrales volumétricos de ventana fija
│   ├─ risk/             # A1: score de riesgo decaído por host
│   ├─ sigma/            # A4: convertidor Sigma determinista
│   ├─ alert/ actions/ webhook/ # alertas, respuestas, conector SIEM
│   ├─ suppress/ lifecycle/   # supresiones con expiración, triaje
│   ├─ api/              # HTTP local, stats/metrics, export, SSE
│   └─ store/            # SQLite opt-in con poda de retención
├─ pkg/model/            # esquema de eventos compartido
├─ sensor/               # sensor ETW en Rust
├─ rules/windows/        # 23 reglas sembradas (6 ficheros)
├─ sequences/ beacons.yaml thresholds.yaml suppressions.example.yaml
├─ web/console/          # consola Next.js 16 + React Bits
├─ web/console-service/  # hub Bun + socket.io
├─ scripts/dev-tests/    # E2E y smokes sobre binarios reales
├─ scripts/windows/      # instalador PS1, servicio, config Sysmon
└─ docs/                # este documento, OpenAPI, assets, actas

```

Ejemplo 3. Estructura del monorepo (v0.2, árbol real del repositorio).

Las convenciones de código se aplican desde el primer commit: formato obligatorio (gofmt y rustfmt), vet y lint en CI, commits convencionales y documentación de decisión viva (este documento, el README y los informes de ronda). Todo paquete recibe pruebas unitarias antes de fusionarse, y la batería de E2E de scripts/dev-tests ejercita los binarios reales de punta a punta para detectar regresiones de detección, no solo de compilación. Los paquetes que introducen estado de larga vida se auditan además con -race y con pruebas de contención de memoria, una convención que el rol de seguridad elevó a estándar del equipo tras las auditorías de los detectores.

7. Roadmap de Desarrollo

El roadmap se organiza en cuatro fases, cada una con un entregable demostrable al cierre. La regla de priorización es explícita: primero el flujo vertical que detecta (tracer bullet), después la anchura de telemetría y la detección avanzada, después la experiencia, y solo al final la extensibilidad. La tabla 7 añade a la planificación original la columna que la v0.1 no podía escribir: el estado real verificado de cada fase a fecha de esta revisión, con 7 de 17 líneas del roadmap interno cerradas y los cuatro paquetes de detección con ciclo de vida completo.

Fase	Objetivos	Estado hoy (v0.2)
1. Core y MVP (Q4 2026)	Tracer bullet end-to-end; sensor básico; reglas YAML puntuales; CLI de prueba.	Aterrizada y medida: pipeline completo, 23 reglas, ingest autenticado, p99 $\approx$ 0,4 ms medido con bench (promesa original: < 10 ms).
2. Detección avanzada (Q1 2027)	Correlación de secuencias; almacenamiento forense; YARA; eBPF para Linux.	Mayormente aterrizada: correlador con cap de estado, store SQLite con retención, beaconing (A3), umbrales (A2) y riesgo (A1). YARA y eBPF siguen pendientes.
3. Experiencia (Q1-Q2 2027)	Consola web con dashboard en vivo; API documentada; conectores Elastic/Splunk.	En curso: consola Next.js de operaciones con triaje, OpenAPI validado por guard en CI, webhook SIEM/SOAR y export JSONL/CSV. Conectores empaquetados pendientes.
4. Extensibilidad (Q2 2027)	Filaments en sandbox Python; profiling; guía de contribución.	Pendiente. Sin código simulado en el árbol; el diseño se mantiene en la sección 3.4.

Tabla 7. Fases del roadmap con su estado real a septiembre de 2026.

Las métricas de éxito miden la salud del proyecto, no solo el código. La fase 1 prometía un pipeline estable con latencia p99 bajo diez milisegundos: el bench nocturno mide hoy 0,4 milisegundos en el peor de los casos medidos sobre loopback, con completitud 2000/2000 de alertas muestreadas en el mismo reloj. Para las fases 2 y 3 siguen en pie los umbrales de cobertura de telemetría, el tiempo de instalación bajo cinco minutos y los primeros usuarios externos emitiendo feedback en el tracker. La estrategia de comunidad acompaña al producto desde el día uno: documentación de arranque en diez minutos, casos de uso por técnica MITRE y una etiqueta good-first-issue siempre abierta.

8. Plan de Acción y Verificación

El tracer bullet que acompañó a la v0.1 sigue siendo la puerta de entrada al proyecto: un disparo vertical completo que valida la arquitectura antes de construir sobre ella. Su valor no es funcional sino de riesgo: demuestra que el esquema de eventos, el protocolo de transporte, el motor de reglas y las alertas encajan, y lo hace hoy, en cualquier máquina con Go instalado, sin depender de Windows. La Figura 3 muestra el flujo de los cinco pasos que ejecuta el andamiaje.

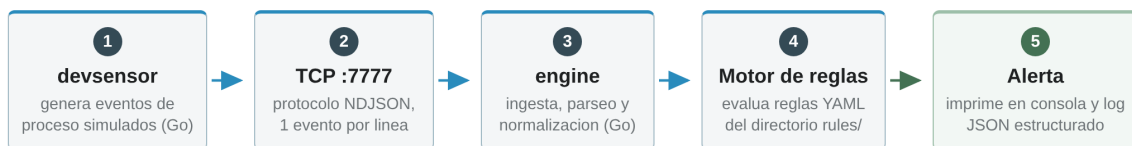


Figura 3. Tracer bullet: pipeline de evento a alerta ejecutable en cualquier plataforma.

8.1 Ejecutar el tracer bullet

El motor arranca en una terminal y escucha por TCP en el puerto 7777; el sensor de desarrollo genera en otra terminal eventos de proceso simulados, algunos benignos y otros emulando PowerShell codificado o descargas con certutil; con `-burst` añade ráfagas de escritura que disparan los umbrales volumétricos. En segundos la consola muestra las alertas disparadas con severidad y la telemetría cruda en formato JSON estructurado. El mismo flujo, cambiando el devsensor por el binario Rust en un host Windows, consume ETW real: el motor no sabe la diferencia porque el contrato es el esquema de eventos. La importación de un pack comunitario cierra el círculo con dos comandos.

```

# 1. Arrancar el motor (terminal 1)
make run-engine
# -> ingest :7777 con AUTH, API :7778, reglas + secuencias + beacons + umbrales

# 2. Generar eventos (terminal 2)
make run-devsensor          # escenario canónico + etiqueta demo
sf-devsensor -burst 60      # rafaga para el detector de umbrales (A2)

# 3. Resultado esperado
# [ALERT] HIGH T1059.001 powershell.exe -enc ... host=LAB-WKS-01

# 4. Importar reglas Sigma de la comunidad
sf-engine sigma -dir sigma-rules/ -out rules/imported/ -strict
  
```

Ejemplo 4. Comandos del tracer bullet y del importador Sigma.

8.2 Verificación continua

La verificación no es una fase final sino parte del ciclo de cada ronda. La batería de scripts/dev-tests ejerce binarios reales de punta a punta: `e2e_sigma` (10/10, incluida la salida estricta con código 1), `e2e_threshold` (12/12, incluido el respeto de supresiones sobre alertas volumétricas), `e2e_beacon` (14/14, incluida la verificación anti-re-vinculación de puertos), `smoke_auth`, `smoke_lifecycle` y `e2e_store_sequences`. El guard del OpenAPI compara el spec con el motor vivo (12 rutas y 29 campos de stats, con paridad exacta contra `/metrics`) y falla la ronda si hay deriva. Los paquetes con estado se testean con `-race`, e `ingest` y `beacon` forman parte de la batería estándar desde la auditoría cruzada del detector de beaconing. El bench nocturno cierra el círculo de rendimiento con presupuesto de latencia explícito.

A partir de aquí, el roadmap del capítulo 7 gobierna las próximas rondas: empaquetar los conectores SIEM como entrega oficial, iniciar el spike de YARA para escaneo de memoria bajo demanda, explorar el collector eBPF para Linux y publicar el release etiquetado v0.2 con notas de versión y demo grabada. La arquitectura descrita en este documento es, sobre todo, una promesa de estabilidad para quien construya encima: el comportamiento del pipeline no cambiará, solo mejorará su implementación, y cada revisión futura de este documento vendrá acompañada de la misma columna de estado honesto que distingue a esta v0.2 de su original.